

Topic 7: Recursion

7.1 What is Recursion?

- A function that calls itself to solve a smaller version of the same problem
- Every recursive solution needs: a base case (terminates recursion) and a recursive case
- Each call pushes a new stack frame -- too many calls causes stack overflow
- Many problems have elegant recursive solutions: tree traversal, divide-and-conquer

7.2 Classic Examples

- Factorial: $\text{fact}(n) = n * \text{fact}(n-1)$; base: $\text{fact}(0) = 1$
- Fibonacci: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$; base: $\text{fib}(0)=0, \text{fib}(1)=1$
- Binary search: compare mid; recurse on left or right half
- Note: naive Fibonacci is $O(2^n)$ -- memoisation reduces it to $O(n)$

```
int factorial(int n) {
    if (n == 0) return 1;           // base case
    return n * factorial(n - 1);    // recursive case
}

int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n-1) + fibonacci(n-2);
}
```

7.3 Recursion vs Iteration

- Any recursive algorithm can be converted to iterative (using an explicit stack)
- Recursion is cleaner for tree/graph traversal, divide-and-conquer
- Iteration is more memory-efficient (no stack frame overhead)
- Tail recursion: recursive call is the last thing done -- compiler can optimise to iteration

7.4 Divide and Conquer

- Divide the problem into smaller sub-problems of the same type
- Conquer: solve sub-problems recursively
- Combine: merge sub-problem results
- Examples: merge sort, quick sort, binary search
- Recurrence relation for merge sort: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ by Master Theorem

```
void mergeSort(int* a, int l, int r) {  
    if (l >= r) return;  
    int mid = (l + r) / 2;  
    mergeSort(a, l, mid);  
    mergeSort(a, mid+1, r);  
    merge(a, l, mid, r);    // combine step  
}
```
